

Тестовое задание: порт экрана регистрации с C/Informix на Python

Контекст

Старая университетская система написана на **C со встроенным SQL (Informix ESQL/C)**. Мы переносим её экраны на Python (frappe framework). В репозитории — один экран целиком, со всеми слоями. Он выполняет три операции над регистрацией студента в семестре:

- **drop** — снять студента с секции;
- **add** — записать на секцию (проверки: hold, окно регистрации, пререквизиты, кредитный лимит, места, лист ожидания);
- **swap** — заменить одну секцию на другую (drop, затем add).

```
legacy/src/   main.c (драйвер) proc.ec (бизнес-логика) db.ec (доступ к БД) dec.h (коды/константы)
legacy/sql/   schema.informix.sql — оригинальная схема (для справки)
data/         schema.sqlite.sql + seed.sqlite.sql — ваш порт работает с этим
```

Компилировать .ec не нужно — это спецификация. Как читать ESQL/C: обычный C, но строки с \$ (\$select , \$insert , \$update , \$delete , \$COMMIT/ROLLBACK WORK) — встроенный SQL; \$var в запросе — host-переменная (в Python — параметр ?); \$char / \$long — типы; результат проверяется через SQLCODE (100 / SQLNOTFOUND — строк нет, <0 — ошибка БД, 0 — успех).

Что нужно сделать

Рабочий end-to-end порт экрана на Python, **поведенчески эквивалентный оригиналу**.

Рамки: Python 3.10+, только стандартная библиотека, БД — sqlite3 поверх

data/*.sqlite.sql . Без внешних зависимостей (pytest допустим как dev-зависимость для тестов). Структуру кода выбираете сами.

Сохраните разделение на три слоя: доступ к данным (db.ec , один запрос на функцию, без бизнес-правил) · бизнес-логика (do_drop / do_add / do_swap , правила и **их порядок**, коды исхода) · тонкий вход (main.c , разбор аргументов, печать сообщения, exit-код).

Обязательно

1. Операции **drop** и **add**, три слоя.
2. CLI-контракт (см. ниже): **дословные** сообщения и exit-коды.
3. Транзакции: при ошибке БД — **полный откат, без частичных изменений**, успех фиксируется.
4. Прохождение сценариев приёма **S1–S13**.

На выбор

- **A. swap** (сценарий S14).

- **A+. Атомарный swap** : вся операция в одной транзакции, при любом сбое — полный откат (S15).
- **B. Автотесты** — основной способ показать корректность. Идеально — покрытие всех сценариев.

CLI-контракт

```
<entrypoint> drop <id> <sec> <yr> <sess>
<entrypoint> add  <id> <sec> <yr> <sess>
<entrypoint> swap <id> <dropsec> <addsec> <yr> <sess>
```

Критерии приёмки

Сценарии выверены на эталонной реализации, на чистой БД из `schema.sqlite.sql` + `seed.sqlite.sql` . **Запускайте каждый на свежей БД.** Блок: Must = обязательно, A / A+ = опции.

#	Блок	Команда	Сообщение	exit
S1	Must	drop 1001 CS101-01 2024 S	Done.	0
S2	Must	drop 1002 CS101-02 2024 S	Done.	0
S3	Must	drop 1003 CS101-01 2024 S	Cannot proceed: student has a registration hold.	1
S4	Must	drop 1004 CS101-01 2024 S	Cannot drop: a grade has already been posted.	1
S5	Must	drop 1001 HX999-01 2023 F	Cannot drop: the drop deadline has passed.	1
S6	Must	drop 1007 CS101-02 2024 S	Student is not enrolled in that section.	1
S7	Must	add 1007 CS101-02 2024 S	Done.	0
S8	Must	add 1007 CS201-01 2024 S	Cannot add: prerequisites are not satisfied.	1
S9	Must	add 1007 PHIL300-1 2024 S	Cannot add: prerequisites are not satisfied.	1
S10	Must	add 1008 PHIL300-1 2024 S	Section full -- student placed on the waitlist.	0
S11	Must	add 1001 MA200-01 2024 S	Student is already enrolled in that section.	1
S12	Must	add 1007 HX999-01 2023 F	Cannot add: registration for this term is closed.	1
S13	Must	add 1007 CS101-01 2024 S	Section full -- student placed on the waitlist.	0
S14	A	swap 1001 CS101-01 CS201-01 2024 S	Done.	0
S15	A+	swap 1001 MA200-01 CS999-01 2024 S	Student is not enrolled in that section.	1